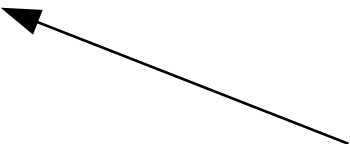


Aggiunta di un pulsante (button)

MainActivity.java

Assicurarsi che il file *MainActivity.java* contenga solo le seguenti istruzioni:

```
package it.miodominio.contaclick;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```



identificatore del file
activity_main.xml che si trova
nella cartella /res/layout

Il file *activity_main.xml*

- Un **file di layout** è un file xml che descrive l'aspetto grafico di una schermata o di una sua parte (analogo ad HTML)
- Specifica la disposizione (layout) dei componenti nella pagina...

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

- Per definire la nostra schermata, cancelliamo il contenuto del file e lo sostituiamo col codice mostrato a pagina seguente.

Modifichiamo il file *activity_main.xml*

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#4CAF50"
    android:padding="20dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Ancora nessun click"
        android:textSize="25dp"
        android:textStyle="bold"
        android:textColor="#ffffff"/>

    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="CLICCA QUI"/>

</LinearLayout>
```

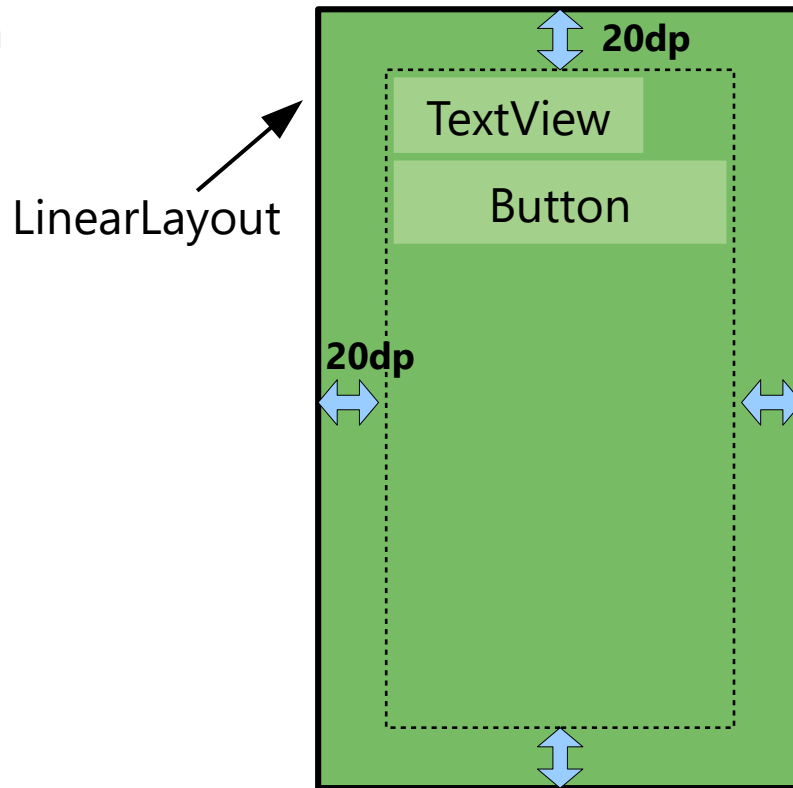


Osservazioni sul file di layout

- Nel file sono presenti tre *componenti grafici*: un **LinearLayout**, una **TextView**, e un **Button**. Ad ognuno di essi corrisponde un *elemento* xml con diversi *attributi*.
- Gli elementi appartengono al namespace <http://schemas.android.com/apk/res/android>
- L'elemento **LinearLayout** è un contenitore (simile al div in HTML) che dispone i propri componenti figli in verticale (cioè uno sotto l'altro) oppure in orizzontale (uno di fianco all'altro) in base al valore dell'attributo *orientation*.

Osservazioni sul file di layout

- l'attributo padding aggiunge un margin interno di 20dp al LinearLayout.



L'unità di misura *dp*

- "dp" significa *Density-independent Pixel* ed è l'unità di misura più usata in Android.
- 1dp corrisponde ad 1 pixel sugli schermi a media densità (160dpi) mentre corrisponde ad un numero maggiore di pixel sugli schermi ad elevata densità. In questo modo, ad esempio, un pulsante lungo 100dp mantiene circa la stessa lunghezza (in millimetri) su schermi diversi.
- Ogni elemento deve avere una larghezza e un'altezza impostate tramite gli attributi *layout_width* e *layout_height*. Tali attributi sono obbligatori.

larghezza e altezza

Ogni elemento deve obbligatoriamente avere una larghezza e un'altezza impostate tramite gli attributi ***layout_width*** e ***layout_height***.

Il valore di questi attributi può essere:

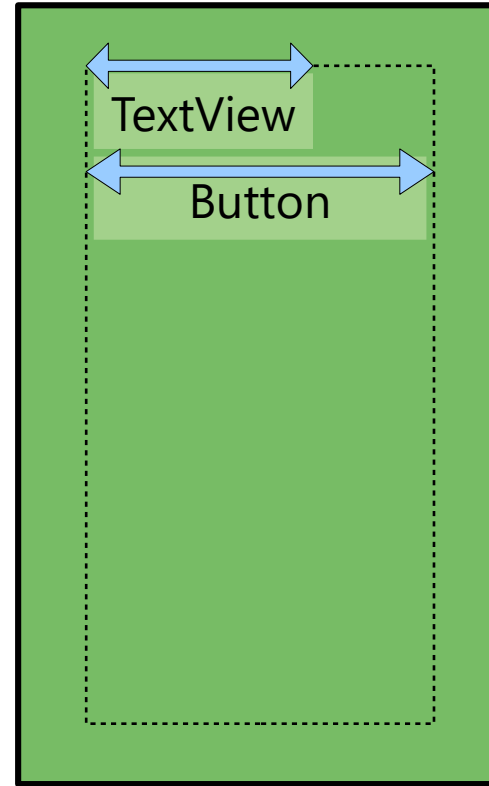
- un numero con unità di misura (generalmente si usa *dp* ma esistono altre unità)
- uno dei due valori speciali *match_parent* oppure *wrap_content*

match_parent indica che il componente deve espandersi fino ad occupare tutto lo spazio disponibile nel componente padre

wrap_content indica che il componente deve occupare lo spazio minimo necessario per racchiudere ("avvolgere") il proprio contenuto

Larghezza e altezza dei componenti

- La TextView occupa solo lo spazio necessario a racchiudere il testo in essa contenuto, poichè è stato specificato il valore "wrap_content" per la larghezza
- Il Button invece occupa tutto lo spazio disponibile in larghezza poichè è stato specificato il valore "match_parent"



Aggiunta di un ascoltatore

- Ora dobbiamo aggiungere il codice da eseguire alla pressione del pulsante. Un blocco di codice eseguito al verificarsi di un evento su un componente grafico è detto **ascoltatore**
- Dobbiamo modificare due file:
 - Nel file *MainActivity.java* aggiungiamo un metodo con un nome a scelta e con questa forma (fissa): **public void nomeMetodo(View v) { ... }**
 - Nel file *activity_main.xml* specifichiamo l'associazione tra pulsante e il metodo tramite l'attributo **onclick**, e assegnamo un identificatore alla casella di testo tramite l'attributo **id**.

activity_main.xml dopo le modifiche

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#4CAF50"
    android:padding="20dp">
```

```
    <TextView
        android:id="@+id/contatore"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Ancora nessun click"
        android:textSize="25dp"
        android:textStyle="bold"
        android:textColor="#ffffff"/>
```

Aggiunta di un ID a nostra scelta
dopo la sequenza fissa **@+id/**

```
    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="CLICCA QUI"
        android:onClick="pressioneTasto1"/>
```

Nome del metodo Java da
richiamare (deve essere un metodo
pubblico, con tipo ritorno void e
con parametro di tipo View)

```
</LinearLayout>
```

MainActivity.java dopo le modifiche

```
package it.miodominio.contaclick;
import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.view.View;
import android.widget.*;

public class MainActivity extends AppCompatActivity {
    private int conta;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    public void pressioneTasto1(View v) {
        Toast.makeText(this, "Tasto premuto", Toast.LENGTH_SHORT).show();
        TextView tv = findViewById(R.id.contatore);
        conta++;
        tv.setText( "Hai cliccato " + conta + " volte ");
    }
}
```

Osservazioni sul codice

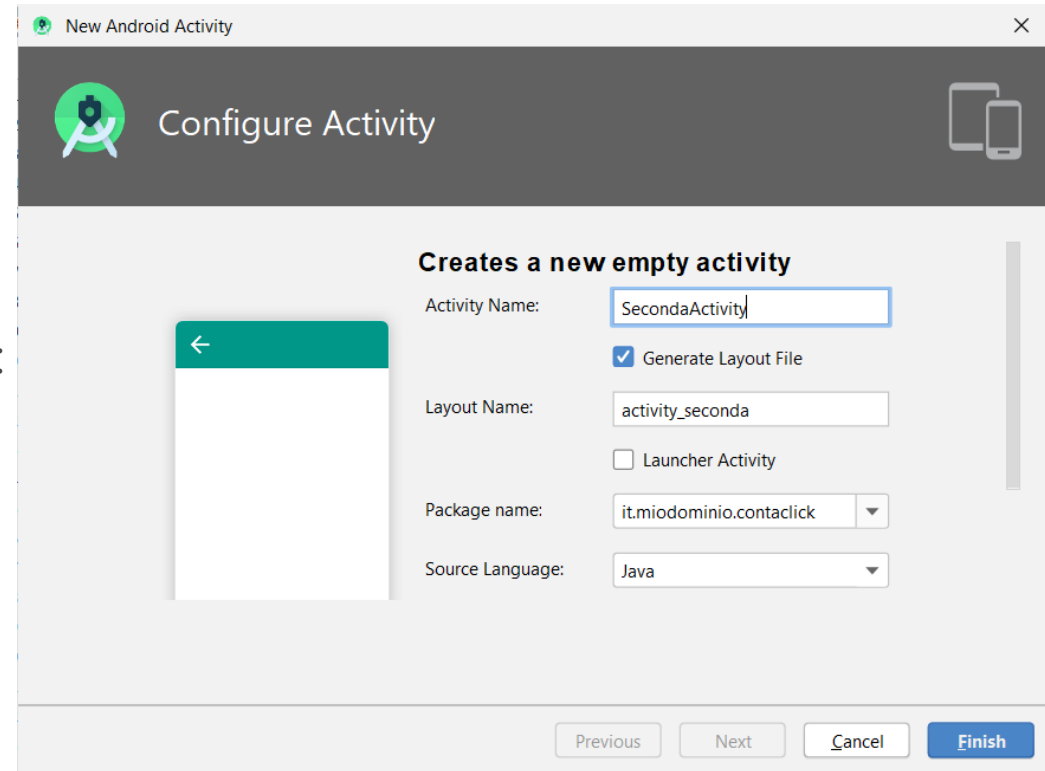
Domanda: perchè il metodo prevede un parametro di tipo View?

Risposta:

- Ad ogni componente grafico corrisponde una classe (esistono ad esempio le classi Button, TextView, LinearLayout)
- La classe **View** è la classe base da cui discendono tutte queste classi, quindi ogni oggetto della classi elencate è anche una View
- Nei metodi che fungono da ascoltatore l'argomento di tipo View costituisce un riferimento al componente grafico che ha generato l'evento (nel nostro caso il bottone che ha generato il "clic")
- Ovviamente non è obbligatorio usare il riferimento se non ci serve (come nell'esempio)

Aggiunta della seconda Activity

- Per aggiungere una nuova activity scegliere **File | New | Activity | Empty view Activity**
- Scegliendo i nomi come in figura Android studio genererà in automatico:
 - il nuovo file **SecondaActivity.java**
 - il nuovo file **activity_seconda.xml**
 - un nuovo elemento **<activity> </activity>** nel file manifest.xml



manifest.xml dopo l'aggiunta dell'activity

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="it.miodominio.contaclick">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".SecondaActivity"></activity>
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Ogni nuova activity deve essere dichiarata nel manifest



Impostiamo il layout della nuova activity

- Scriviamo questo codice nel file *activity_seconda.xml*

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="#4CAF50">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="App realizzata da\nPinco Pallino"
        android:textSize="30dp"
        android:textAlignment="center"
        android:layout_gravity="center"
        android:background="#80FFFFFF" ←
        android:padding="30dp"
        android:layout_margin="10dp"/>

</LinearLayout>
```

Se si esprime il colore tramite 8 cifre esadecimali, le prime due cifre specificano il canale alpha, ovvero il livello di trasparenza/opacità (00 = completamente trasparente, FF = completamente opaco). In questo caso il colore è un bianco (FFFFFF) con opacità 0x80

SecondaActivity.java

SecondaActivity.java deve contenere quanto segue:

```
package it.miodominio.prova1;

import ...

public class SecondaActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_seconda);
    }
}
```

Avviare la seconda activity

Per avviare la nuova activity creiamo il seguente metodo nel file *MainActivity.java* e aggiungiamo import ***android.content.Intent***

```
public void pressioneTasto2(View v) {  
    Intent intent = new Intent(this, SecondaActivity.class);  
    startActivity(intent);  
}
```

e inseriamo un nuovo pulsante nel file *activity_main.xml*

```
<Button  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:text="Informazioni"  
    android:onClick="pressioneTasto2"/>
```

Spiegazione del codice

- La classe **Intent** (in italiano "intento") è una classe fondamentale del framework
- Un oggetto di questa classe può essere pensato come un **messaggio** che può essere inviato al sistema operativo per richiederogli di "fare qualcosa".
- Un intent rappresenta l'**intenzione** di svolgere un'azione/avviare un'attività.
- L'istruzione **Intent intent = new Intent(this, SecondaActivity.class)** crea quindi un messaggio che rappresenta *"l'intenzione di avviare la seconda activity"*.
- Il primo parametro del costruttore è il riferimento all'activity che ha originato il messaggio
- con l'invocazione del metodo **startActivity** (della classe Activity) il messaggio viene passato al sistema operativo che "esaudisce" la richiesta.

Risultato finale

